MCA 1st Semester

CS7034: Full Stack Web Development using MERN

Practical Workbook

Set-A

Course Objective: Provide foundational knowledge of modern full stack web application development using MERN stack.

Course Outcomes:

Upon completion of the course student shall be able to:

CO1: Acquire knowledge in modern web development technologies using MERN stack, and advanced JavaScript concepts.

CO2: Implement advanced ES6 features to write efficient and scalable code.

CO3: Build interactive user interfaces using React and state management effectively.

CO4: Implement backend services using Express.js to facilitate client-server communication via HTTP.

CO5: Utilize JavaScript's asynchronous capabilities for data handling and server-side operations.

CO6: Manage data manipulation and storage on MongoDB server using the Node application.

Programme Outcomes:

PO1: Proficiency in and ability to identify problems related to computer science as well as design and apply computational knowledge to solve them.

PO2: Ability to design, develop, test and maintain system, component, product or process as per needs and specification.

PO3: Understanding of professional and ethical role and responsibility.

PO4: Recognition of the need for and ability towards life-long learning.

PO5: Knowledge of programming languages, database systems, operating systems, software engineering, Web & Mobile technology and relevant modern issues.

PO6: Ability to demonstrate the use of modern tools, models and languages to solve problems related to software development.

PO7: Ability to communicate and present knowledge effectively.

Semester Skills:

SO1: Thinking Skill: Critical

SO2: Learning Skill: Solitary(Intrapersonal)

SO3: Listening Skill: Evaluative

SO4: Writing Skill: Expository

SO5: Courses' based technical skills

Number of Practical Problems in Journal

Unit	No. of Practical Problems	Time required to implement (in hours)	Minimum required for journal evaluation
1	02	06	02
2	02	08	02
3	02	08	02
4	02	08	02
5	02	08	02
6	02	08	02
Total:	**12**	**46**	**12**

Tools and Technologies:

- Any IDLE (Preferable Visual Studio Code)
- Node.js and npm
- MongoDB or MongoDB Atlas
- React JS, Babel, and Express.js

Suggested Learning Resources:

1. Vasan Subramanian – "Pro MERN Stack Full Stack Web App Development with Mongo, Express, React, and Node", Apress.
2. https://www.javascripttutorial.net/es6/
3. https://legacy.reactjs.org/docs/getting-started.html
4. https://nodejs.org/en
5. https://www.mongodb.com/
6. https://expressjs.com/

<table>
<tr><td>Practical No.: 1</td><td colspan="2"></td><td colspan="2">Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="4">To understand the features of ES6</td></tr>
<tr><td>Practical Exercise:</td><td colspan="4">Develop a Contact Manager functionality by applying ES6 features as given below:

i. Define a function to create a new contact. If no phone number is provided, it should default to "Not Provided".
ii. Use the rest parameter to allow adding multiple phone numbers for a contact.
iii. Use the spread operator to merge existing contacts with newly added contacts.
iv. Using for...of loop, iterate over the list of contacts to display their details.
v. Format the contact details into a string using template literals.

<u>Sample Output:</u>

Contact List:
Name: Nisha Patel
Email: nisha.patel@yahoo.com
Phone Numbers: Not Provided

Name: Ammar Shaikh
Email: ammar.shaikh@gmail.com
Phone Numbers: 9876543210

Name: Isha Mistry
Email: isha.mistry@utu.ac.in
Phone Numbers: 9123456789, 9123456790

Name: Nisha Patel
Email: nisha.patel@yahoo.com
Phone Numbers: 9988776655
---------------------------</td></tr>
<tr><td>Duration for Completion:</td><td>03</td><td colspan="3"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="4">https://www.javascripttutorial.net/es6/</td></tr>
<tr><td>Solution must contain:</td><td colspan="4">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="5">Evaluation Parameters: 1. Function with default parameter and Template literals 2. Rest Parameter and Spread Operator, 3. For...of Loop, 4. Technical via</td></tr>
<tr><td>Marks:</td><td>2</td><td>4</td><td>2</td><td>2</td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Teacher's Signature:</td><td colspan="4"></td></tr>
<tr><td>Date:</td><td colspan="4"></td></tr>
</table>

<table>
<tr><td>Practical No.: 2</td><td colspan="3"></td><td>Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="4">To understand Higher-order function, callbacks and arrow function.</td></tr>
<tr><td>Practical Exercise:</td><td colspan="4">A. Create a higher-order function named “processArray” that performs a specified operation on each element of an array. The “processArray” function takes two parameters:
a. An array of elements.
b. A callback function that defines the operation to be performed on each element.
Callback functions that can be used with “processArray” are:
i. double: A function that doubles its input.
ii. triple: A function that triples its input.
iii. square: A function that squares its input.
iv. cube: A function that cubes its input.

<u>Sample Output:</u>
<pre>const numbers = [1, 2, 3, 4, 5];

Doubled Numbers: [2, 4, 6, 8, 10]
Tripled Numbers: [3, 6, 9, 12, 15]
Squared Numbers: [1, 4, 9, 16, 25]
Cubed Numbers: [1, 8, 27, 64, 125]</pre>
B. Use map() to apply the operation function to each element.

C. Conver all functions created in ‘A’ to an arrow function.</td></tr>
<tr><td>Duration for Completion:</td><td>03</td><td colspan="3"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="4">https://www.javascripttutorial.net/es6/</td></tr>
<tr><td>Solution must contain:</td><td colspan="4">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="5">Evaluation Parameters: 1. Higher-order function using ‘function’ keyword and name 2. Usage of map() and conversion to arrow function 3. Technical viva</td></tr>
<tr><td>Marks:</td><td>4</td><td>4</td><td>2</td><td></td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Teacher’s Signature:</td><td colspan="4"></td></tr>
<tr><td>Date:</td><td colspan="4"></td></tr>
</table>

<table>
<tr><td>Practical No.: 3</td><td colspan="2"></td><td colspan="2">Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="4">To create and render React elements and components and to apply validations.</td></tr>
<tr><td>Practical Exercise:</td><td colspan="4">Develop a Farm Management Dashboard to manage farm produce and farmer details.

A. Create a <code>Farmer</code> functional component that renders a farmer's name, age, and farm location. Pass the farmer's details as props from the parent component ‘App’.

B. Create a FarmProduct functional component that renders the product name, type (e.g., vegetable, fruit, and grain), and quantity available. These details should also be passed as props from App.

C. Use Property Validation: Implement Zod to validate the props passed to both Farmer and FarmProduct components.
i. The farmer’s name should be a string.
ii. The farmer’s age should be a number and mandatory.
iii. The farm location should be an optional string.
iv. The product’s quantity should be a number and required, also create a custom validation that minimum quantity 1 and quantity do not exceed 10.
v. Create validation for the product type to ensure it belongs to either 'vegetable', 'fruit', or 'grain'.

D. Render Multiple Farmers and Products, Inside the App component, render at least five farmers and ten farm products by passing props to each component.</td></tr>
<tr><td>Duration for Completion:</td><td>04</td><td colspan="3"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="4">https://legacy.reactjs.org/docs/react-component.html,
https://legacy.reactjs.org/docs/typechecking-with-proptypes.html</td></tr>
<tr><td>Solution must contain:</td><td colspan="4">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="5">Evaluation Parameters: 1. Usage functional component with props 2. Validation 3. Rendered multiple data 4. Technical viva</td></tr>
<tr><td>Marks:</td><td>3</td><td>4</td><td>2</td><td>1</td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Teacher’s Signature:</td><td colspan="4"></td></tr>
<tr><td>Date:</td><td colspan="4"></td></tr>
</table>

<table>
<tr><td>Practical No.: 4</td><td colspan="2"></td><td>Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="3">To implement Styled Components and React Routing</td></tr>
<tr><td>Practical Exercise:</td><td colspan="3">Consider Practical No-3, and extend the Farm Management Dashboard by adding style to the components and a navigation system.

A. Apply dynamic styling to components. [There is no restriction on style; students can use styles other than the specified ones to earn bonus marks.]
i. The Farmer component should have a styled card layout with a background color and padding.
ii. The FarmProduct component should display the product in a box with a different background based on the type of product such as vegetable: green, fruit: red, and grain: yellow.

B. Implement React Router to navigate between a Farmers page and a Products page.
i. On the Farmers page, display the list of farmers (from Practical-3). Similarly, the Products page should show the farm products.
ii. Create a Home page with a brief welcome message about the farm management system and add a navigation menu with links to the Farmers and Products pages. [Students can design the Home page based on their own creativity.]</td></tr>
<tr><td>Duration for Completion:</td><td>04</td><td colspan="2"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="3">https://styled-components.com/docs/basics#styling-any-component, https://reactrouter.com/en/main/start/tutorial</td></tr>
<tr><td>Solution must contain:</td><td colspan="3">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="4">Evaluation Parameters: 1. Usage of styled component 2. Usage of React routing 3. Technical viva</td></tr>
<tr><td>Marks:</td><td>04</td><td>04</td><td>02</td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td></tr>
<tr><td>Teacher's Signature:</td><td colspan="3"></td></tr>
<tr><td>Date:</td><td colspan="3"></td></tr>
</table>

<table>
<tr><td>Practical No.: 5</td><td colspan="3"></td><td>Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="4">To implement the concepts of React State Management and Form</td></tr>
<tr><td>Practical Exercise:</td><td colspan="4">Create a component FarmInventoryManager that manages a list of farm products. Fetch product data asynchronously from an API available at URL https://api.jsonbin.io/v3/b/671a0377acd3cb34a89c2c5f (Use setTimeout to access a remote JSON file).

A. Initialize the state with an empty product list and then asynchronously populate the list after the data is fetched.

B. Use the useState hook to manage the products and useEffect to fetch the data when the component is first mounted.

C. Display the fetched farm products details (except photos) on the page in formatted manner once they are loaded.

D. Provide functionality to add new product entry, for that create a form to allow Farmers to add a new product to the inventory. The form should have all fields specified in API except photos.

E. Handle form submissions using React's controlled components, upon form submission, update the inventory list with the new product as well as populate newly entered product on browser.

[<u>Note</u>: While designing form, use appropriate HTML elements according to the nature of fields. Do not use only textboxes.]</td></tr>
<tr><td>Duration for Completion:</td><td>04</td><td colspan="3"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="4">https://react.dev/reference/react-dom/components/form
API URL - https://api.jsonbin.io/v3/b/671a0377acd3cb34a89c2c5f</td></tr>
<tr><td>Solution must contain:</td><td colspan="4">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="5">Evaluation Parameters: 1. Asynchronous fetch 2. Usage of useState & useEffect hooks 3. React Form to add new product details 4. Display data using child component</td></tr>
<tr><td>Marks:</td><td>2</td><td>3</td><td>3</td><td>2</td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Teacher's Signature:</td><td colspan="4"></td></tr>
<tr><td>Date:</td><td colspan="4"></td></tr>
</table>

<table>
<tr><td>Practical No.: 6</td><td colspan="2"></td><td colspan="2">Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="4">To implement the concept of Context API in React.</td></tr>
<tr><td>Practical Exercise:</td><td colspan="4">Implement a context API that holds the state for farmer information, including: Farmer ID, Name, Contact, Farm Location, List of Farm Products (grains, vegetables, fruits). Ensure that all components consuming the context reflect updates in real-time without needing to pass data through props.

A. Build a Provider component that wraps around your main application or specific parts of it, allowing components to access and update the farmer's state.

B. Create a FarmerProfile component that displays the farmer's profile information. This component should consume the context to render the farmer's details dynamically.

C. Build a component that lists all farm products associated with the farmer. This component should also consume the context and render the production data dynamically.

D. Create a form component that allows farmers to add new production details to their list (consider all fields mentioned in above given API for Farm products). This should also update the context state.

E. Implement a feature that allows users to remove production details from the list. Ensure that this updates the context state accordingly.

<u>Intermediate Level (Optional):</u> Implement a form component that allows farmers to update their personal information (contact information and farm location). The form should consume the context to modify the farmer's data.</td></tr>
<tr><td>Duration for Completion:</td><td>04</td><td colspan="3"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="4">https://react.dev/learn/passing-data-deeply-with-context</td></tr>
<tr><td>Solution must contain:</td><td colspan="4">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="5">Evaluation Parameters: 1. Context creation 2. Context provider 3. Context consuming 4. Technical viva.</td></tr>
<tr><td>Marks:</td><td>4</td><td>2</td><td>2</td><td>2</td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Teacher’s Signature:</td><td colspan="4"></td></tr>
<tr><td>Date:</td><td colspan="4"></td></tr>
</table>

Practical No.: 7		**Enrolment No.**
Objective:	To understand the concept of building RESTful API using Express JS.	

Practical Exercise:	Build a RESTful API in Express.js that manages farmer details for APMC Market to manage farmer wise crop. The API will use the provided JSON structure as a resource and enable CRUD operations for farmers and their produced crops. [Note: Create and use the JSON file in given structure to perform read/write operations.]
	
	1. Add a New Farmer - Route: POST /farmers
	• Accept details for a new farmer (including farmer ID, name, contact, etc). Add the new farmer to the JSON file with an empty crop list.
	• Validate that the farmer's field includes name, phone number and an email.
	• Return a success message with the added farmer details.
	2. List All Farmers and Their crops - Route: GET /farmers
	• Fetch and display the list of all farmers along with their produced crops.
	3. Retrieve Details of a Specific Farmer - Route: GET /farmers/:farmerId
	• Retrieve the details of a specific farmer, including their list of produced crops.
	• Return an error message if the farmer ID does not exist.
	4. Enter Farmer wise crops - Route: POST /farmers/:farmerId/crops
	• Add a new crops to a specific farmer's crop list using the farmer ID.
	• Validate the crop details (e.g., required fields like name, quantity, price, and category).
	• Ensure the category is one of the predefined categories (Grains, Vegetables, Fruits).
	• Return the updated farmer-crops assignment details after the operation.
	5. Remove a crop from a Farmer - Route: DELETE /farmers/:farmerId/crops/:cropId
	• Remove a specific crop from a farmer's crops list using farmer and crop IDs.
	• Return a success message confirming the removal.
	6. Delete a Farmer - Route: DELETE /farmers/:farmerId
	• Remove a farmer and all their assigned crops from the JSON file.
	• Return a success message confirming the deletion.
	7. Create Error Handling Middleware provide meaningful error messages for requests whenever required.
	
	JSON File Format:
	
	`{`
	`"farmers": [`
	`{`
	`"farmerId": 1,`
	`"name": "Ramesh Patel",`
	`"contact": {`
	`"phone": "9876543210",`
	`"email": "ramesh.patel@example.com"`
	`},`
	`"farmLocation": "Ahmedabad, Gujarat",`
	`"producedCrops": [`	

```
        {
          "cropId": 101,
          "cropName": "Wheat",
          "category": "Grains",
          "quantity": 1200,
          "unit": "kg",
          "pricePerUnit": 25,
          "harvestSeason": "Winter"
        },
        {
          "cropId": 102,
          "cropName": "Tomato",
          "category": "Vegetables",
          "quantity": 500,
          "unit": "kg",
          "pricePerUnit": 40
        }
      ]
    },
    {
      "farmerId": 2,
      "name": "Suresh Mehta",
      "contact": {
        "phone": "9988776655",
        "email": "suresh.mehta@example.com"
      },
      "farmLocation": "Nashik, Maharashtra",
      "producedCrops": [
        {
          "cropId": 201,
          "cropName": "Rice",
          "category": "Grains",
          "quantity": 1500,
          "unit": "kg",
          "pricePerUnit": 30,
          "harvestSeason": "Monsoon"
        },
        {
          "cropId": 202,
          "cropName": "Mango",
          "category": "Fruits",
          "quantity": 300,
          "unit": "dozen",
          "pricePerUnit": 120
        }
      ]
    }
```

	] }		
Duration for Completion:	**04**		
Reference to solve the problem:	https://expressjs.com/en/guide/using-middleware.html		
Solution must contain:	Written description for the given problem and printout of output screen.		
Evaluation Parameters: *1. Implementation of given Routes, 2. Error-handling Middleware, 3. Technical viva*			
Marks:	**6**	**2**	**2**
Secured by Student:			
Teacher's Signature:			
Date:			

<table>
<tr><td>Practical No.: 8</td><td colspan="3"></td><td>Enrolment No.</td></tr>
<tr><td>Objective:</td><td colspan="4">To apply the concept of API consumption using fetch().</td></tr>
<tr><td>Practical Exercise:</td><td colspan="4">Consume API created in Practical-7 in React App using fetch(), and do as directed.

1. Validation: Implement validation for incoming data to ensure integrity such as required fields and valid formats for each datatypes.
2. Create one Form for Farmer Entry.
3. Create another form for Farmer wise crops entry.
• Provide dropdown for Farmer selection.
• Dynamic form fields for crop category.
o The crop category dropdown is rendered with three options: Grains, Vegetable and Fruits.
o When a user selects 'Grains', the condition formData.cropsCategory === "Grains" evaluates to true, triggering the rendering of additional fields (harvest season).
o If "vegetables" or "fruits" is selected, these fields will not be rendered.
4. Create Controlled components for better form handling.
5. State management for real-time updates.
6. Display Farmer wise crops list in formatted manner.</td></tr>
<tr><td>Duration for Completion:</td><td>04</td><td colspan="3"></td></tr>
<tr><td>Reference to solve the problem:</td><td colspan="4">https://expressjs.com/en/guide/using-middleware.html,
https://react.dev/reference/rules/rules-of-hooks</td></tr>
<tr><td>Solution must contain:</td><td colspan="4">Written description for the given problem and printout of output screen.</td></tr>
<tr><td colspan="5">Evaluation Parameters: 1. Consumption of API using fetch(), 2. Creation of two forms with validation, 3. Data Entry using two forms, 4. Display Data from API.</td></tr>
<tr><td>Marks:</td><td>02</td><td>02</td><td>04</td><td>02</td></tr>
<tr><td>Secured by Student:</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Teacher's Signature:</td><td colspan="4"></td></tr>
<tr><td>Date:</td><td colspan="4"></td></tr>
</table>

Practical No.: 9		**Enrolment No.**		
Objective:	To understand the Database connectivity and CRUD operations.			
Practical Exercise:	Consider Practical No. 7 and modify the API routes to perform CRUD operations in MongoDB using the Mongoose library. Create the required schema and model with data validation to define the structure of the data. Also test the built API using any API testing tool.			
Duration for Completion:	**04**			
Reference to solve the problem:	https://www.mongodb.com/developer/languages/javascript/getting-started-with-mongodb-and-mongoose/			
Solution must contain:	Written description for the given problem and printout of output screen.			
Evaluation Parameters: *1. Database connectivity 2. Creation of Schema and Model 3. CRUD Operations 4. Technical viva*				
Marks:	**02**	**03**	**04**	**01**
Secured by Student:				
Teacher's Signature:				
Date:				

Practical No.: 10		**Enrolment No.**			
Objective:	To utilize the third-party libraries of front end in React App.				
Practical Exercise:	Consume the API created in Practical No. 9, in a React application and provide the following functionalities: • Utilize appropriate React component libraries for efficient UI design, such as Material UI, React Bootstrap, React Redux, and Semantic UI. 1. Add a new farmer. 2. List all farmers with details, while on clicking a specific farmer display his/her added crops. 3. Retrieve details of a specific farmer. 4. Add crops for a specific farmer. 5. Remove a crop from a farmer. 6. Delete a farmer.				
Duration for Completion:	**04**				
Reference to solve the problem:	https://react-bootstrap.netlify.app/				
Solution must contain:	Written description for the given problem and printout of output screen.				
Evaluation Parameters: *1. Usage of UI Designing library 2. Efficient appearance 3. Working of given Functionalities 4. Technical Viva*					
Marks:	**2**	**3**	**4**	**1**	
Secured by Student:					
Teacher's Signature:					
Date:					